INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

UNIDAD DE APRENDIZAJE: BASES DE DATOS

PRÁCTICA 1: MODELO ENTIDAD-RELACIÓN

Ejercicio 4
Estado del arte: artículos científicos

Profesor: Hurtado Avilés Gabriel

Integrantes

Damas Garnica Adrian

Esparza Ramón Fernando Adán

Ramos Padrón Jesus Emmanuel

Hernandez Barrios Samuel Rodrigo

Tercer semestre

Grupo: 3CV4

Fecha: 18 de septiembre de 2026

Ejercicio 4. Estado del Arte: Optimización de Bases de Datos mediante Aprendizaje Automático

Artículo 1

Cita (APA 7): Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Polyzotis, N. (2018). The case for learned index structures. En Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18) (pp. 489–504). Association for Computing Machinery. https://doi.org/10.1145/3183713.3196909

Problema: Las estructuras de índice tradicionales de un SGBD —árboles B, tablas hash, bitmaps — se diseñan de forma genérica, sin aprovechar que la distribución real de las llaves almacenadas suele seguir un patrón predecible, lo que desperdicia memoria y ciclos de cómputo en cada búsqueda.

Método: Los autores proponen tratar un índice como un modelo de aprendizaje automático que predice directamente la posición de un registro dentro de un arreglo ordenado, y presentan el Recursive Model Index (RMI), una jerarquía de modelos simples en cascada que se especializan en distintos rangos de datos.

Resultado: Los índices aprendidos superan a los árboles B optimizados en caché hasta en un 70% en velocidad de búsqueda, y reducen el consumo de memoria en un orden de magnitud frente a estructuras tradicionales sobre conjuntos de datos reales.

Conexión con la Unidad 1: Corresponde al módulo de gestión de almacenamiento dentro de los módulos componentes del SGBD (tema 1.2.1) y a la independencia física de datos (tema 1.3.2), porque demuestra que la estructura interna de acceso puede reemplazarse por completo sin que el usuario perciba el cambio a nivel lógico o externo.

Aporte a mi proyecto: La tabla reproduccion de mi modelo crecerá de forma continua con cada escucha registrada; si el volumen de registros se vuelve muy grande, un índice aprendido sobre la columna timestamp_inicio podría acelerar los reportes por rango de fechas sin necesidad de rediseñar el esquema relacional.

Artículo 2

Cita (APA 7): Marcus, R., Negi, P., Mao, H., Zhang, C., Alizadeh, M., Kraska, T., Papaemmanouil, O., & Tatbul, N. (2019). Neo: A learned query optimizer. Proceedings of the VLDB Endowment, 12(11), 1705–1718. https://doi.org/10.14778/3342263.3342644

Problema: Los optimizadores de consultas tradicionales dependen de reglas y estimadores de costo escritos a mano por ingenieros, que se degradan cuando cambia la forma de los datos o el patrón de las consultas, y cuyo mantenimiento exige experiencia muy especializada.

Método: Neo sustituye por completo al optimizador de consultas por una red neuronal entrenada con aprendizaje por refuerzo, que aprende a construir planes de ejecución directamente a partir de la retroalimentación de ejecuciones reales, sin depender de reglas predefinidas.

Resultado: Con menos de 24 horas de entrenamiento sobre cargas de trabajo industriales, Neo iguala o supera el rendimiento de optimizadores comerciales maduros, aunque los propios autores reconocen que su desempeño se degrada frente a cargas de trabajo muy dinámicas.

Conexión con la Unidad 1: Se relaciona con el procesador de consultas dentro de los módulos del SGBD (tema 1.2.1) y con el lenguaje DML (tema 1.2.2), ya que interviene directamente en cómo se ejecutan las sentencias SELECT que emiten los usuarios de la base de datos.

Aporte a mi proyecto: Mi motor de recomendación depende de consultas que cruzan cancion, reproduccion y lista_cancion; un optimizador que aprenda los patrones reales de esas consultas podría mejorar el tiempo de respuesta del sistema sin que yo tenga que ajustar manualmente cada consulta a mano.

Artículo 3

Cita (APA 7): Marcus, R., Negi, P., Mao, H., Tatbul, N., Alizadeh, M., & Kraska, T. (2021). Bao: Making learned query optimization practical. En Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21) (pp. 1275–1288). Association for Computing Machinery. https://doi.org/10.1145/3448016.3452838

Problema: Los enfoques previos de optimización basados en aprendizaje automático, como Neo, requieren descartar por completo el optimizador existente y entrenar un reemplazo desde cero, lo cual implica una sobrecarga de entrenamiento alta y poco práctica para sistemas ya en producción.

Método: Bao conserva el optimizador tradicional del SGBD y solo aprende a elegir, entre un conjunto reducido de sugerencias de ejecución (hints), cuál aplicar a cada consulta, combinando redes neuronales convolucionales sobre árboles con muestreo de Thompson, un algoritmo clásico de aprendizaje por refuerzo.

Resultado: Bao aprende estrategias útiles en menos de una hora de entrenamiento, mejora el rendimiento de punta a punta —especialmente la latencia de cola en consultas de larga duración — y en entornos de nube reduce tanto costos como tiempos de respuesta frente a sistemas comerciales; el trabajo obtuvo el Best Paper Award de SIGMOD 2021.

Conexión con la Unidad 1: Extiende el mismo tema de módulos del SGBD (tema 1.2.1) que el Artículo 2, pero además ejemplifica la arquitectura cliente-servidor en la nube (tema 1.3.3), pues sus resultados se validan sobre motores comerciales alojados como servicio.

Aporte a mi proyecto: A diferencia de entrenar un optimizador completo desde cero, un enfoque tipo Bao —aprender solo a elegir entre un puñado de configuraciones ya probadas— es mucho más realista para un proyecto académico como el mío, corriendo sobre PostgreSQL en un contenedor Docker con recursos y tiempo limitados.

Comparación y Discusión

Los tres artículos comparten la misma tesis de fondo: que componentes centrales de un sistema gestor de bases de datos —tradicionalmente diseñados con reglas fijas y estructuras genéricas— pueden reemplazarse o asistirse con modelos de aprendizaje automático que se ajustan a los datos y consultas reales de cada instalación. Kraska et al. (2018) sientan la base teórica al mostrar que un índice puede entenderse, literalmente, como un modelo; Marcus et al. (2019) llevan esa idea al extremo al sustituir el optimizador de consultas completo; y Marcus et al. (2021) corrigen el problema práctico que deja Neo,

mostrando que basta con aprender a elegir entre un conjunto reducido de decisiones ya validadas por el optimizador tradicional.

La diferencia principal entre los tres está en cuánto del sistema original reemplazan: el primero sustituye una sola estructura de datos, el segundo sustituye el optimizador entero, y el tercero conserva el optimizador tradicional y solo aprende a dirigirlo, lo que explica por qué Bao es el único de los tres pensado explícitamente para desplegarse sobre motores comerciales en producción. El problema abierto que comparten, y que identifico como el más relevante para cualquier sistema que dependa de estos enfoques, es la falta de garantías: un índice o un optimizador aprendido puede fallar de forma impredecible ante datos que se alejan de la distribución con la que fue entrenado y, a diferencia de un árbol B o un optimizador basado en reglas explícitas, no existe una forma sencilla de auditar por qué tomó una decisión específica, lo cual es delicado en un sistema que, como el que yo diseñé, maneja datos de comportamiento real de usuarios.